Video script — MARCO Crystallisation Triage (target: 7:45, hard cap 8:00)

Shooting script for the coursework demo video. Each block has a target duration, what to have on screen, and narration to read/paraphrase in your own words — don't read it robotically, use it as talking points. Numbers below are pulled directly from `docs/report.md`, `experiments/log.csv` and `docs/genai_evaluation.md` — verified against those files, not invented for the video.

Before recording — have these open in tabs/windows ready to switch to, so there's no dead time hunting for files: 1. Terminal, repo root 2. `notebooks/01_eda.ipynb` (rendered, scrolled to the class/source distribution charts) 3. `notebooks/02_baseline.ipynb` (scrolled to the confusion matrix / PR curve) 4. `notebooks/03_ann_initial.ipynb` and `notebooks/06_ann_final.ipynb` (learning curves for both) 5. `docs/report.md`'s "ANN: Initial and Final Models (Q2, Q3)" section (the side-by-side search/learning-curve figures) and "Model Comparison (Q3)" section (the comparison table) 6. Browser at `http://127.0.0.1:8000` (confirm the backend is running — see below) 7. `notebooks/04_xai.ipynb` (Grad-CAM/occlusion/LIME comparison figure) 8. `docs/genai_evaluation.md`'s comparison table 9. The three ready-to-upload misclassified test images in `data/demo_errors/` — the false negative (`01_crystals_false_negative.png`) is the one used live in the demo below

Start the backend before recording:

```
PYTHONPATH=src uvicorn app.backend.main:app --host 127.0.0.1 --port 8000
```

0:00–0:35 — Intro (0:35)

Show: face-cam or title slide with project name.

> *Hi, I'm Chamika Subashinie. This is my MSc Data Science coursework project. I built a model that sorts protein crystallisation drop images into four classes: Clear, Crystals, Precipitate and Other, using the MARCO dataset.*
>
> *The aim is to help a crystallographer decide which drops to look at first. Only drops with crystals are worth harvesting and sending to a synchrotron beamline, so for this user, missing a real crystal is much more costly than a false alarm. That imbalance guided every modelling decision I made.*

0:35–[TBD, ~1:40] — Data download (~1:05) — TIMING NOT YET ADJUSTED, see note at bottom

Show: open `scripts/download_extract.py` in the editor; `notebooks/00_download_explore.ipynb`'s per-shard and pooled chi-square cells; `data/splits.csv` head.

> *The full MARCO dataset is large: about 463,000 images, around 55 gigabytes, stored as TFRecord files split into shards. The training part alone has 407 shards of about 1,000 images each.*
>
> *To understand the data, I first downloaded one shard and counted how many images were in each of the four classes. Then I wanted to know: do all the shards have a similar class mix? So I downloaded two more shards from different parts of the dataset and compared them.*
>
> *The share of Crystals images was close in all three shards: 11.8, 13.2 and 14.8 percent. The full dataset has 12.8 percent. One shard alone can be a little off, but together the three shards matched the full dataset well.*
>
> *This told me that a sample made from several shards would represent the whole dataset, so I used 15 shards, about 14,400 images, for this project. This is the script that downloads these 15 shards and extracts the images.*

EDA — TIMING NOT YET SET (content only, per instruction)

Show: `01_eda.ipynb` — leakage-grouping cells, `scripts/make_splits.py`, the chi-square cell, then the class distribution / source distribution / image-size charts.

> *Next, the exploratory data analysis.*
>
> *Leakage control. The same well is often photographed more than once, and a well can hold several drops. If those images ended up in both training and testing, the test score would look better than it really is. So I read the plate and well numbers from the filenames and gave every image a group key. Images from the same plate and well share a key.*
>
> *Splitting the data. I then split the data into training, validation and test sets, roughly 70, 15 and 15 percent, using scikit-learn's StratifiedGroupKFold. "Group" means whole groups stay together, so no well crosses a split. "Stratified" means each set keeps a similar class balance. I saved the split as a CSV file with each image's ID, label, group key and split. Every model uses this same file, and I can check at any time that no group crosses a split.*

> *Is the sample representative? On the training and validation images, I ran a chi-square test against the class counts of the full MARCO dataset. There was no significant difference, so my sample reflects the population.*
>
> *Exploring the data. I plotted the class distribution, the source institutions, and the image sizes for each class. Image sizes vary widely, which is why every image is resized before training.*

Classical baseline: LightGBM — TIMING NOT YET SET (content only, per instruction)

Show: `02_baseline.ipynb` — confusion matrix and PR curve; optionally the feature-importance chart from `docs/figures/q3_baseline_feature_importance.png`.

> *Classical baseline. Before any neural network, I built a classical model to compare against. Each image is turned into hand-crafted features: HOG, which describes edges and shapes, and LBP, which describes texture. A LightGBM model then learns to classify those features. I chose LightGBM over XGBoost and SVM because it trains quickly on almost 2,000 features and handles class imbalance well. I used balanced class weights, so the rare Crystals class counts as much as the others.*
>
> *Setting the precision rule. On the validation set, I plotted the baseline's precision–recall curve for Crystals. From it I set my rule: at least 30 percent of the drops the model flags must really contain crystals. That means out of every 10 flagged drops, at least 3 are real crystal drops. I fixed this level before training any neural network and used it unchanged for every model, so it can't favour any of them.*
>
> *Result. On the test set, the baseline found 93.7 percent of the crystal drops at this precision. It trained in under 30 seconds, so it's a strong and fast model to beat.*

Initial ANN — TIMING NOT YET SET (content only, per instruction)

Show: `03_ann_initial.ipynb` learning curves and the new failed-experiment figure (`q3_ann_initial_failure.png`); terminal showing `train_ann.py --help` or the hyperparameter search bar chart.

> *Initial ANN. For the neural network, I used a ResNet backbone pretrained on ImageNet. I kept the backbone frozen and trained only a new head on top: a dropout layer and a final layer for my four classes.*
>
> *Hyperparameter search. I searched the three required hyperparameters: learning rate, from 0.0003 to 0.01; batch size, 16, 32 or 64; and network depth, ResNet18 or ResNet34. That's 24 possible combinations. I ran a random search of 12 configurations, each trained for 12 epochs. Because each setting was drawn*

> *independently, one combination came up twice, so the search covered 11 different combinations.*
>
> *Where it ran. Training on my laptop was too slow, so I needed a GPU. I started on Google Colab's free tier, but it kept cutting me off partway through the search. A paid subscription was an option, but I only needed a few hours of GPU time, so renting a GPU on RunPod by the hour was cheaper and more reliable. The training code is exactly the same either way. Only the machine changed.*
>
> *What the learning curves showed. Training and validation loss both dropped in the first epoch, then flattened out, and stayed close together. So the model wasn't overfitting, but it also stopped improving. With the backbone frozen, only about two thousand numbers in the head can learn, and they sit on top of features that were never adapted to drop images. That plateau is what led to my next step: the Final ANN.*
>
> *A failed experiment. One setting failed: the highest learning rate, 0.01, with the smallest batch size, 16. Training became unstable. The loss went up instead of down, and crystal recall dropped to 0.62. I kept it in my log rather than hiding it. It showed that this model needs a smaller learning rate.*

3:20–4:30 — Final ANN: 4th hyperparameter, comparison table, error analysis (1:10)

Show: the side-by-side search/learning-curve figures in `docs/report.md`'s "ANN: Initial and Final Models (Q2, Q3)" section first, then its "Model Comparison (Q3)" section for the table (or the PDF pages).

> *Final ANN. The plateau told me the frozen backbone was the limit. So I added a fourth hyperparameter: degree of fine-tuning. I compared keeping the backbone frozen with unfreezing its last block, layer4, so the model could adapt its top-level features to crystallisation images instead of reusing ImageNet features as they are.*
>
> *You can see the effect in these two charts. Across the same 12 configurations, the Final ANN scores higher than the Initial ANN. And its training loss keeps falling across all 12 epochs instead of flattening after the first, because it now has room to learn.*
>
> *Test results. Here is the final comparison on the held-out test set. Each model was tested exactly once.*

Model	recall@P_MIN (primary)	Crystals PR-AUC	Accuracy
Trivial benchmark (majority class)	0.000	0.124	0.480

Model	recall@P_MIN (primary)	Crystals PR-AUC	Accuracy
Classical baseline (LightGBM, HOG+LBP)	0.937	0.678	0.812
Initial ANN (frozen)	0.886	0.672	0.715
Final ANN (layer4 unfrozen)	**0.965**	**0.823**	0.809

> *The trivial benchmark always guesses the most common class. It gets 48 percent accuracy but finds no crystals at all, which shows why accuracy alone is a poor measure here.*
>
> *The classical baseline finds 93.7 percent of crystal drops.*
>
> *The frozen Initial ANN finds 88.6 percent. Interestingly, it beat the baseline on validation but lost to it on test. I report that honestly.*
>
> *The Final ANN, with layer4 unfrozen, finds 96.5 percent: the best result. Its test score matched its validation score, so the improvement is real, not just tuned to the validation set.*
>
> *So my pre-registered expectation, that the ANN would win, was correct, but only after I unfroze part of the backbone.*

> *I also did a proper error analysis on the Final ANN's test errors, not just the aggregate number. Breaking accuracy down by source institution shows real spread — 60 to 69% on the two smallest, most underrepresented sources, versus up to 85% on the best-performing one — directly confirming the EDA's source-imbalance finding on held-out data, not just a hypothesis. And Precipitate is the single biggest source of confusion for every other class, on both the baseline and the ANN — so that's a dataset-level pattern, not something specific to one architecture.*

4:30–6:10 — Live demo: web app, real-time prediction, Grad-CAM, incorrect prediction (1:40)

Show: browser at `http://127.0.0.1:8000`. Upload a normal, correctly-classified test image first (e.g. a clear Crystals example), then later in this section upload the false-negative image from `data/demo_errors/` (see below) — the required incorrect-prediction demo now lives in this section, not a separate one.

> *This is the web app — FastAPI backend, simple frontend. Every prediction here runs the actual trained model fresh, right now, nothing is precomputed. I'll upload a crystal image.*

[Upload image, let it run, point at the result]

[Point at the heatmap]

> *Explanation feature. This heatmap is Grad-CAM. It shows which part of the image most influenced the prediction. I compared it with two other methods. Occlusion covers small patches of the image one at a time and measures how much the prediction changes. LIME splits the image into regions and tests which ones matter most. Grad-CAM is the only one fast enough to run live, so it's the one in the app. The other two I used offline to check it. For a crystallographer, Grad-CAM shows where to look. It doesn't prove how the model reasons.*
>
> *I checked that it's trustworthy in two ways. First, when I covered the highlighted area, the model's Crystals score fell from 0.62 to 0.14, so that area really drives the prediction. Second, when I reset the model's trained layers to random weights, the heatmap changed completely. So it reflects what the model learned.*

[Upload the false-negative image — `data/demo_errors/01_crystals_false_negative.png` — required: at least one incorrect prediction shown live]

> *A real mistake. This drop really contains crystals, but the model gives it only a 0.8 percent chance of Crystals. That's the costly error: a missed crystal. All three methods point at the centre of the drop, so the model is looking in the right place but reading the texture wrong. More training examples like this would likely help more than changing the model.*

6:55–7:35 — GenAI comparison (0:40)

Show: `docs/genai_evaluation.md` comparison table.

> *Finally, I compared all of this against a pretrained vision-language model — Claude Haiku 4.5 — zero-shot, no training on MARCO at all, evaluated on the exact same test images with the same metric. It did **worse than every trained model, and worse than the trivial majority-class benchmark**.*

7:35–7:55 — Wrap-up (0:20)

Show: face-cam or the comparison table again.

> *To sum up: I built a fixed split that keeps images from the same well together, so there's no leakage. A classical baseline set a high bar. The neural network beat it only after I unfroze part of the backbone. Grad-CAM shows where the model looks,*

> *and I checked that it's trustworthy. And a general AI model, with no training on this task, did far worse, which shows why training on real crystallisation data still matters.*
>
> *The predictions in the app ran live, with nothing precomputed. Thank you for watching.*

Timing budget check

STALE — not yet updated for the longer Data download narration below. Data download grew from ~15s to ~1:05 (147+ words). Everything from Data download onward in this table is wrong until the next retime pass; total is currently well over 8:00. Content correctness was prioritised first per instruction — do not record from these timestamps yet.

Section	Duration	Running total
Intro	0:35	0:35
Data download	~1:05 (was 0:15)	~1:40 (stale)
EDA	0:45	1:35
Classical baseline	0:40	2:15
Initial ANN + RunPod	1:05	3:20
Final ANN + comparison + error analysis	1:10	4:30
Live demo (prediction + Grad-CAM + sanity checks)	1:40	6:10
Incorrect prediction (required)	0:45	6:55
GenAI comparison	0:40	7:35
Wrap-up	0:20	7:55

Leaves ~5s buffer under the 8:00 cap. This now explicitly covers all four checklist items: error analysis (Final ANN section), request-time prediction (live demo), a sanity-checked XAI method (both required sanity checks named with their actual numbers), and an incorrect prediction analysed with XAI (required: 1 in video, 3 in the report's "Error Predictions and Analysis (Q3, Q5)" section). If you run long in practice, trim "Data download" or "Wrap-up" first — not the live demo, the incorrect prediction, or the sanity-check lines, since those are what the checklist is actually looking for.